



Ransomware (Malware)

Forensics, Malware and Penetration Testing

Anivedh Auradkar

Faheem Rahman

Reedwan Ali

Zain Ul-Abideen

Cyber Security MSc

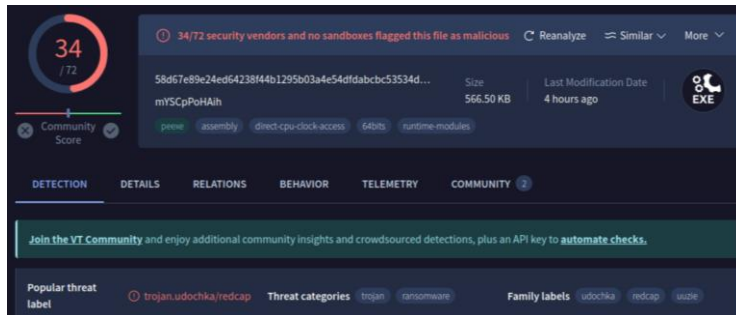
March 2024

Table of Contents

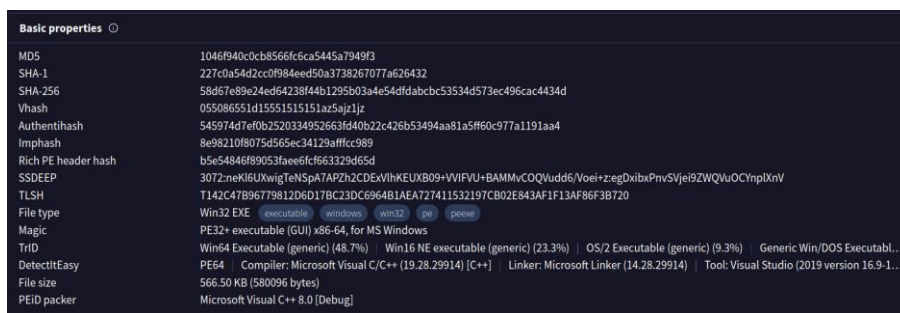
1. Static Analysis.....	2
2. Dynamic Analysis.....	3
3. Reverse Engineering & source code analysis	3
4. Directories targeted by malware	4
5. Decrypting Hank's backup files.....	4
6. Appendix.....	5

1. Static Analysis

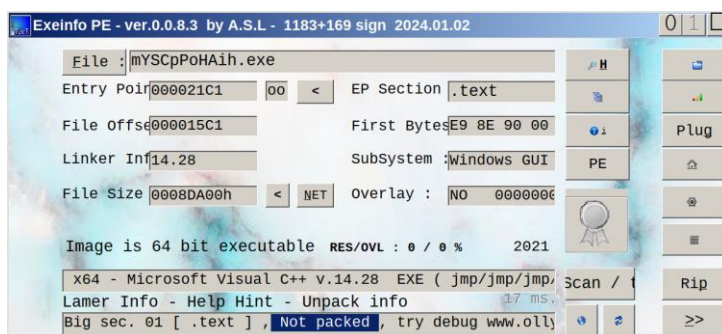
It is important to gauge an initial understanding of the malware and file itself using a variety of analysis tools which can be used to identify and verify the malware's core behaviour and features.



VirusTotal scores it 34/72, labelling it as the possible ransomware 'trojan.udochka/redcap'.¹



Analysing its basic properties reveals it to be Win32bit executable file with a SHA-1 of '227c0a54d2cc0f984eed50a3738267077a626432' and SHA-256 of '58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d'.²



¹ <https://www.virustotal.com/gui/file/58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d/detection>

² <https://www.virustotal.com/gui/file/58d67e89e24ed64238f44b1295b03a4e54dfdabcbc53534d573ec496cac4434d/details>

It is important to verify whether the file is packed or not which has been done by using the program Exeinfo PE in which the file has been verified to not be packed thus making it far less likely to avoid detection.

2. Dynamic Analysis

We run the malware file in a Windows 10 sandbox to observe the behaviour and the suspicious connections that were made. (Refer Figure 5)

On running the file as exe in the sandbox, we see the file gets executed and then wipes itself out of the memory. We make use of procmon to observe what processes did the malware alter/create. On observing the process tree, we see the three processes viz. Cmd.exe, conhost.exe and PING.EXE.

Upon further analysis, it was observed that the following dll files were loaded:

ntdll.dll, kernel32.dll, KernelBase.dll, shlwapi.dll, msvcrt.dll, sechost.dll, rpcrt4.dll, bcrypt.dll, kernel.appcore.dll

Another noteworthy point is, the following registry key whose result indicates that data that is being read or written to/from this registry key is larger than the buffer allocated for it. This occurs if the data stored in the registry key exceeds the expected size, potentially due to corruption or unexpected changes which validates the fact that there was some corruption that has taken place.

Operation	Path	Result
RegQueryValue	HKCU\Control Panel\Desktop\PreferredUILanguages	BUFFER OVERFLOW
RegQueryValue	HKCU\Control Panel\Desktop\MuiCached\MachinePreferredUILanguages	BUFFER OVERFLOW

3. Reverse Engineering & source code analysis

The malware was disassembled using Ghidra. In Ghidra we start our analysis with the entry function since we couldn't find a main function. In the entry function, we see a few methods being called out of which we enlist the critical ones in this section which roughly determine the flow of the malware.

The malware first fetches the current working directory and checks the files present in the current directory. (Refer Figure 6)

Another crucial function here is the encrypt function [renamed for better readability] which gives us the way the malware encrypts the files listed in the current directory (Refer Figure 7)

The program then uses FindFirstFileW function to iterate through all the files in the current working directory and then starts encrypting the files one by one. (Refer Figure 8)

If one observes the getcwd function, the listing_files function gets called first proceeding with a sleep of 10000 ms and then the outbound_conn function is executed. Let's observe the outbound_conn function in Figure 9. The outbound_conn function executes a ping command to the IP '127[.]1[.]247[.]73' with a specified threshold of 1 ICMP request and a timeout of 1123 ms and redirects the output (stdout) to Nul, effectively suppressing any output to the console. Further on, it executes the Del command to delete the file named mYSCpPoHAih.exe [name of the malware file].

4. Directories targeted by malware

After analysing the running malware, we noticed that it operates exclusively within the directory where it's located. By reverse engineering the code using Ghidra, we were able to examine the malware's code and understand its directory and encryption methods. This became evident when the ransomware scanned and encrypted files only within its execution directory, excluding system core directories or any others beyond its own. This restricted the ransomware's impact to a specific scope. Since the ransomware targeted local directories, our analysis enabled us to recover backup files later on. The image [Refer Figure 10] shows the encryption process when the malware was run on Hanks files again (just for testing purposes).

5. Decrypting Hank's backup files

The final task for us to complete was the decryption of Hank's files. The first thing we needed to do is to identify the type of encryption that is used, something that we would be able to see on Ghidra. Eventually, we were able to find out that the type of encryption that is used is AES-128 with CBC mode (shown in Figure 1). Once this was discovered, we were also able to recover the key that is used in the encryption, both by looking at Ghidra. Figure 11 shows the recovered key that had been used.

Using this information, we were able to successfully obtain the files that have been used; the folder containing all the files is also attached to this document. A couple of the files can be seen here as well as the key that was used for the decryption. This is shown in Figure 2 and 3.

6. Appendix

```
u_Starting_CBC_encryption_140071218
unicode      u"Starting CBC encryption.\n"
```

Figure 1: The use of CBC encryption here.



Figure 2: Decrypted Picture 1



Figure 3: Decrypted Picture 2

PE-bear_v0.6.7.3 [F:\home\yboxuser\downloads\PE-bear_0.6.7.3_qt4_x86_win_w10\mYScpPoHaih.exe]

File Settings View Compare Info

mYScpPoH...

DOS stub

NT Headers

Signature

File Header

Optional ...

Section Hea...

Sections

EP = 1...

.data

.data

.pdata

.data

.Dcfg

.RDATA

.reloc

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	00	00	00	00	10	00	00	00	00	00	00	00	00	00	00	00
0010	18	04	09	00	...	00	00	00	00	00	00	00	00	00	00	00
0020	A0	08	00	...	48	00	00	00	00	00	00	00	00	00	00	00
0030	40	09	00	...	08	00	00	...	E0	07	00	38	00	00	00	00
0040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0050	00	00	00	00	00	00	00	F0	E0	07	00	38	01	00	00	00
0060	00	00	00	00	00	00	00	00	00	09	00	18	04	00	00	00
0070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

DisasmGeneralStringsDOS HdrRich HdrFile HdrOptional HdrSection Hdrs...

File info

Path

Z:\home\yboxuser\Downloads\PE-bear_0.6.7.3_qt4_x86_win_w10\mYScpPoHaih.exe

Is Truncated?

No

File size

580096

Loaded size

580096

File Alignment Units

1133

ImpHash

8e98210f8075d565ec34129affcc989

Rich Header Hash

b5e54846f89053face6fcf663329d65d

Checksum

94145

MD5

1046f940c0cb8566cf6ca5445a7949f3

SHA1

227c0a54d2cc0f984ced50a3738267077af626432

Found signatures

Offset	Name	Signature	Section
15C1	Microsoft Visual C++ V8.0 (Debug) e...		.text

property	value
footprint...	DB2FE96B5109F423
size	0xC0 (192 bytes)
entropy	4.968
file_ratio	0.02 %

Process Name (PID)	Process Path	Type / Info - www.microsoft.com/en-us/download/details.aspx?id=90820	
JMScPcPolHah.exe (4760)	C:\Users\WAGUJ... 90820FA7-0023-4...	"C:\Users\WAGUJityAccount\Downloads\malware_sample\JMScPcPolHah.exe"	3/21/2023
cmd.exe (6232)	Windows Comm... C:\Windows\SYS... Microsoft Corporat...	90820FA7-0023-4... cmd.exe /ping 127.1.247.3 -n 1 w 1123 & Net & "C:\Users\WAGUJityAccount\Downloads\mal..."	3/21/2023
Conhost.exe (1000)	Console Window H... C:\Windows\Syste... Microsoft Corporat...	90820FA7-0023-4... %?%C:\Windows\system32\conhost.exe &&ffffffffff ForceV1	3/21/2023
PING.EXE (6706)	TCP/IP Ping Comm... C:\Windows\syste... Microsoft Corporat...	90820FA7-0023-4... ping 127.1.247.3 -n 1 -w 1123	3/21/2023

```

1
2 void getcwd(void)
3
4 {
5     undefined auStack584 [32];
6     WCHAR local_228 [264];
7     unsigned long long local_18;
8
9     local_18 = DAT_140086038 ^ (unsigned long long)auStack584;
10    printf((char *)L"Getting current directory.\n");
11    GetCurrentDirectoryW(0x104, local_228);
12    listing_files();
13    Sleep(10000);
14    outbound_conn();
15    thunk_FUN_14000ae80(local_18 ^ (unsigned long long)auStack584);
16    return;
17 }
18

```

```

Decompile: encrypt - (mYScpPoHaih)
67 *local_f0 = local_110[0];
68 BVar1 = WriteFile(local_108,local_f0,4,local_110,(LPOVERLAPPED)0x0);
69 if (BVar1 == 0) {
70     DVar2 = GetLastError();
71     thunk_FUN_140007510(L"Error writing padding size.\n",DVar2);
72     goto LAB_1400073da;
73 }
74 printf((char *)L"Length of file buffer successfully added to file.\n");
75 printf((char *)L"Starting CBC encryption.\n");
76 thunk_FUN_140008790((longlong)local_d8,(longlong)&DAT_140086000,
77     (undefined *) [32]&DAT_140086010);
78 thunk_FUN_140008450((longlong)local_d8,local_100,local_114);
79 printf((char *)L"Successfully encrypted file buffer. Writing to destination file...\n");
80 BVar1 = WriteFile(local_108,local_100,local_114,local_110,(LPOVERLAPPED)0x0);
81 if (BVar1 == 0) {
82     DVar2 = GetLastError();
83     thunk_FUN_140007510(L"Error writing ciphertext.\n",DVar2);
84     goto LAB_1400073da;
85 }
86 } while (local_118 == '\0');
87 thunk_FUN_14003d334(local_f0);
88 local_117 = 1;
89 }
90 }
91 }

```

```

Decompile: listing_files - (mYSCpPoHaih)
5      GetModuleFileNameW((HMODULE)0x0,local_860,0x104);
6      thunk_FUN_14000c700((undefined (*) [32])local_ea0,(undefined (*) [32])local_10c4,6);
7      iVar2 = wcsncmp(local_ea0,L"-en");
8      if (iVar2 != 0) {
9          _Str2 = PathFindFileNameW(local_860);
10         iVar2 = wcsncmp(local_10c4,_Str2);
11         if (iVar2 != 0) {
12             thunk_FUN_140007bc0(local_650,0x104,in_stack_00000000);
13             local_1110 = local_650;
14             thunk_FUN_140007b20(local_1110,0x104,(short *)&DAT_140070fd8);
15             thunk_FUN_140007b20(local_1110,0x104,local_10c4);
16             encrypt(local_1108,local_1110);
17             DeleteFileW(local_1108);
18         }
19     }
20     BVar3 = FindNextFileW(local_1118,(LPWIN32_FIND_DATAW)local_10f0);
21     while (BVar3 != 0) {
22         thunk_FUN_140007bc0(local_e90,0x104,in_stack_00000000);
23         thunk_FUN_140007b20(local_e90,0x104,L"\\-en*");
24         local_1118 = FindFirstFileW(local_e90,(LPWIN32_FIND_DATAW)local_10f0);
25         if (local_1118 == (HANDLE)0xffffffffffffffff) {
26             DVar1 = GetLastError();
27             thunk_FUN_140007510(L"No file found\n",DVar1);
28         }
29         else {
30             printf((char *)L"Preparing to rename files...\n");
31             do {
32                 if ((local_10f0._0_4_ & 0x10) == 0) {
33                     thunk_FUN_140007bc0(local_440,0x104,in_stack_00000000);
34                     local_10f8 = local_440;
35                     thunk_FUN_140007b20(local_10f8,0x104,local_10c4);
36                     thunk_FUN_14000cf50((undefined (*) [16])local_c80,0,0x208);
37                     //thunk_FUN_14000cf50((undefined (*) [16])local_c80,0,0x208);
38                 }
39             } while (1);
40         }
41     }

```

Figure 8 - listing_files function

```

Decompile: outbound_conn - (mYSCpPoHaih)
9      _STARTUPINFOW local_6a8;
10     WCHAR local_638 [264];
11     WCHAR local_428 [520];
12     ulonglong local_18;
13
14     local_18 = DAT_140086038 ^ (ulonglong)&stack0xffffffffffff8e8;
15     lVar1 = 0x68;
16     p_Var2 = &local_6a8;
17     while (lVar1 != 0) {
18         lVar1 = lVar1 - 1;
19         *(undefined *)&p_Var2->cb = 0;
20         p_Var2 = (_STARTUPINFOW *)((longlong)&p_Var2->cb + 1);
21     }
22     lVar1 = 0x18;
23     p_Var3 = &local_6c8;
24     while (lVar1 != 0) {
25         lVar1 = lVar1 - 1;
26         *(undefined *)&p_Var3->hProcess = 0;
27         p_Var3 = (_PROCESS_INFORMATION *)((longlong)&p_Var3->hProcess + 1);
28     }
29     GetModuleFileNameW((HMODULE)0x0,local_638,0x104);
30     thunk_FUN_140007a50(local_428,0x208,
31         (wchar_t *)L"cmd.exe /C ping 127.1.247.73 -n 1 -w 1123 > Nul & Del \"%s\"",
32         local_638);
33     CreateProcessW((LPWSTR)0x0,local_428,(LPSECURITY_ATTRIBUTES)0x0,(LPSECURITY_ATTRIBUTES)0x0,0,
34         0x80000000,(LPVOID)0x0,(LPWSTR)0x0,(LPSTARTUPINFOW)&local_6a8,
35         (LPPROCESS_INFORMATION)&local_6c8);
36     CloseHandle(local_6c8.hThread);
37     CloseHandle(local_6c8.hProcess);
38     thunk_FUN_14000ae80(local_18 ^ (ulonglong)&stack0xffffffffffff8e8);
39     return;
40 }
41

```

Figure 9 - outbound_conn function

Name	Date modified	Type
~en8FKbHWWY	3/22/2024 7:38 PM	JPEG image
~en14	3/22/2024 7:38 PM	Microsoft Edge PDF ...
~enart - Copy	3/22/2024 7:38 PM	Bitmap image
~enart	3/22/2024 7:38 PM	Bitmap image
~enbalance-sheet.xlsx	3/22/2024 7:38 PM	XLSX File
~enbuy	3/22/2024 7:38 PM	JPEG image
~encats	3/22/2024 7:38 PM	MP4 Video
~encompany_party	3/22/2024 7:38 PM	PNG image
~endesign4	3/22/2024 7:38 PM	JPEG image
~enmy-journal - Copy	3/22/2024 7:38 PM	Microsoft Edge PDF ...
~enmy-journal - Copy	3/22/2024 7:38 PM	Rich Text Document
~enmy-journal	3/22/2024 7:38 PM	Rich Text Document
~enSampleVideo_1280x720_30mb	3/22/2024 7:38 PM	MP4 Video
mYSCpPoHAih	3/22/2024 7:08 PM	Application

Figure 10 - Files getting renamed by the malware

14007efac	52 53 44	DotNetPd...		
	53 ce c2			
	09 71 19 ...			
- 14007efac	52 53 44 53	ds	"RSDS"	signature
- 14007efb0	ce c2 09 71 19	GUID	7109c2ce-2519-44d5-931...	guid
	25 d5 44 93 15			
	5f a5 9c 05 5d...			
- 14007efc0	01 00 00 00	ddw	1h	age
- 14007efc4	44 3a 5c 55 73	ds	"D: \\Users \\Meehien \\D...	pdbrname
	65 72 73 5c 4d			

Figure 11- Key used for encryption

Statement of good academic conduct

By submitting this assignment, I understand that I am agreeing to the following statement of good academic conduct:

- I confirm that this assignment is **my own work** and I have not worked with others in preparing this assignment.
- I confirm this assignment was written by me and is in my own words, except for any materials from published or other sources which are clearly indicated and acknowledged as such by appropriate referencing.
- I confirm that this work is not copied from any other person's work (published or unpublished), web site, book or other source, and has not previously been submitted for assessment either at the University of Birmingham or elsewhere.
- I confirm that I have not asked, or paid, others to prepare any part of this work for me.
- I confirm that I have read and understood the University regulations on plagiarism
(<https://intranet.birmingham.ac.uk/as/registry/policy/conduct/plagiarism/index.aspx>).